



محاكاة مرورية ثنائية الأبعاد باستخدام Modern OpenGL (C++)

عرض مشروع جامعي تقني يشرح التصميم، التنفيذ، والتفاعل لمحاكاة حركة مرور ثنائية الأبعاد باستخدام
OpenGL 3.3 Core Profile. وGLEW وGLFW

Implementation Steps

: (خطوات بناء المشروع)



* إعداد البيئة : استخدام مكتبات GLFW لإدارة النوافذ و GLEW للوصول لميزات OpenGL.

* بناء المظلات (Shaders): كتابة كود Vertex و Fragment Shader بلغة GLSL.

* تجهيز البيانات : تعريف إحداثيات الأجسام (السيارات، الطريق، الإشارة) وتخزينها في VBOs.

* حلقة الرسم (Render Loop): تحديث مواقع الأجسام ورسمها 60 مرة في الثانية لتحقيق الحركة

Fragment & Vertex Shaders :

Vertex Shader

استقبال الصفات
(position, color,
texcoord).
الإحداثيات، تمرير بيانات
إلى المرحلة اللاحقة.

Fragment Shader

حساب اللون النهائي لكل
بكسل. استخدام
`uniform ourColor`
لتغيير ألوان العناصر أثناء
التشغيل.

Uniforms

قيمة واحدة تُحدَّث دون
إعادة إرسال VBO: ألوان
الإشارات، حالة المصابيح،
ووقت المحاكاة.

مثال مختصر - `uniform vec3 ourColor;` :يسمح بتغيير ألوان السيارات وإشارة المرور في الوقت الحقيقي.



استخدام Uniforms لتغيير الألوان ديناميكياً

ourColor

مثال:

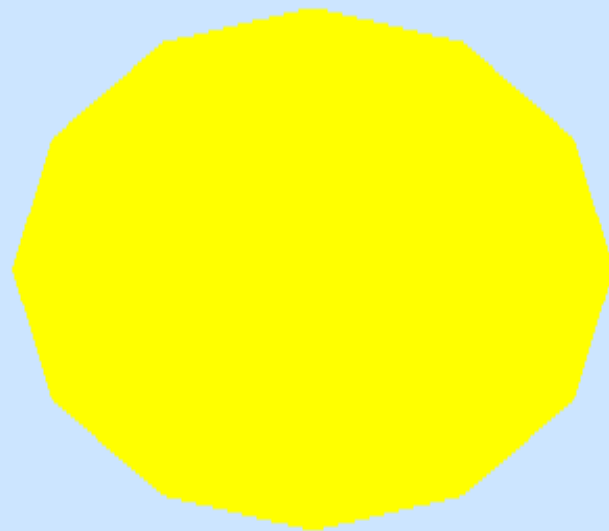
```
glUniform3f(glGetUniformLocation(program,  
"ourColor"), r,g,b); -  
لون السيارة/ضوء الإشارة دون إعادة إرسال  
VBO.
```

Uniforms أخرى

مثل `time, headlightsOn`, و `lightState` تتحكم في مؤثرات الشادر وطقس المشهد.

ميزة الأداء: تحديث حالة العرض بسرعة مع تقليل حركة البيانات عبر PCIe.

Geometry: Triangles & Triangle Fans



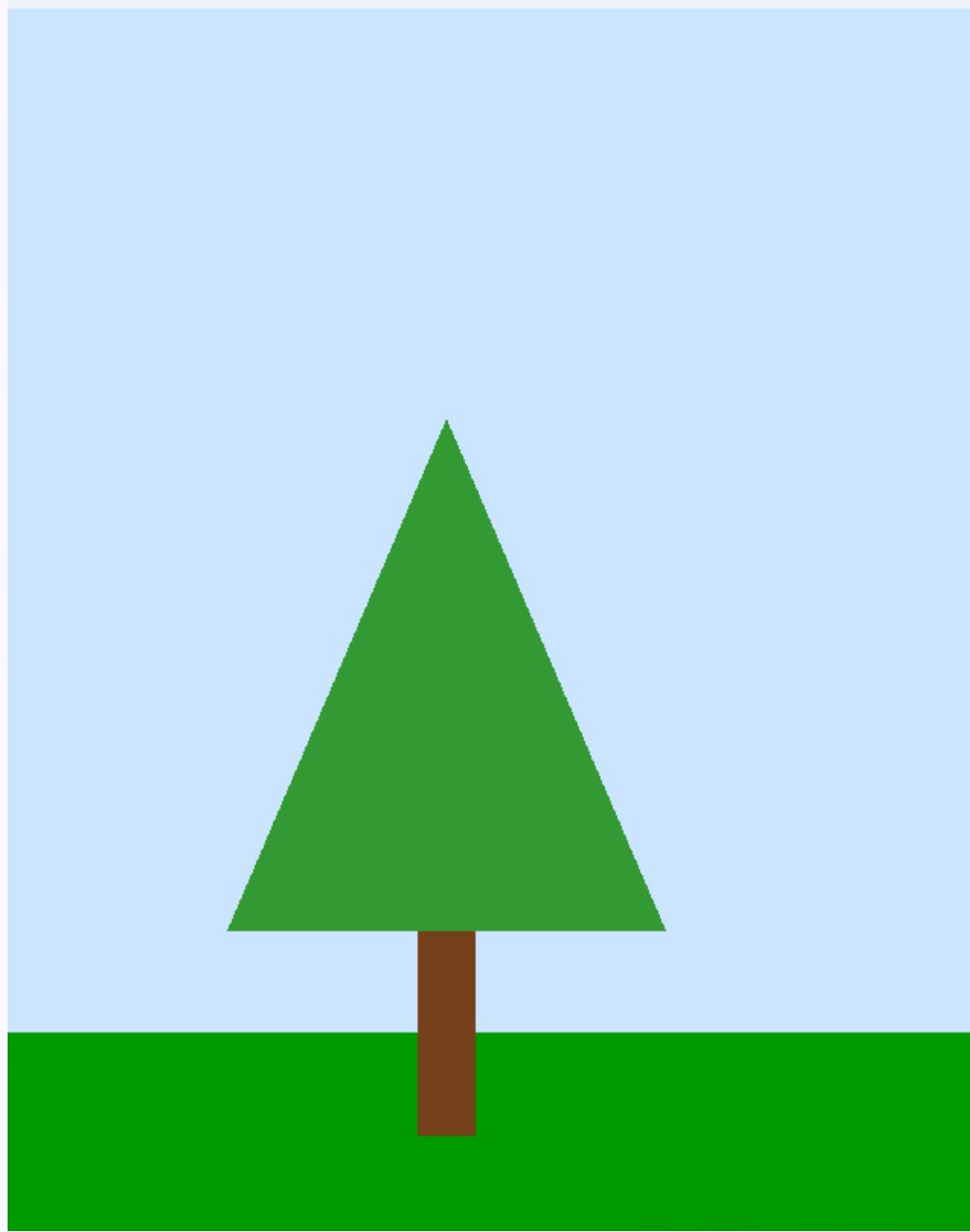
* الأشكال الأساسية : تم بناء جميع الاجسام باستخدام المثلثات `GL_TRIANGLES`

- لرسم الدوائر: ستخدمت دالة `drawCircle` التي تعتمد على

- `GL_TRIANGLE_FAN` لرسم أضواء الإشارة والشمس.

* الدقة: تم تقسيم كل دائرة إلى ١٢ مثلثاً

`CIRCLE_SEGMENTS` لضمان شكل دائري ناعم.



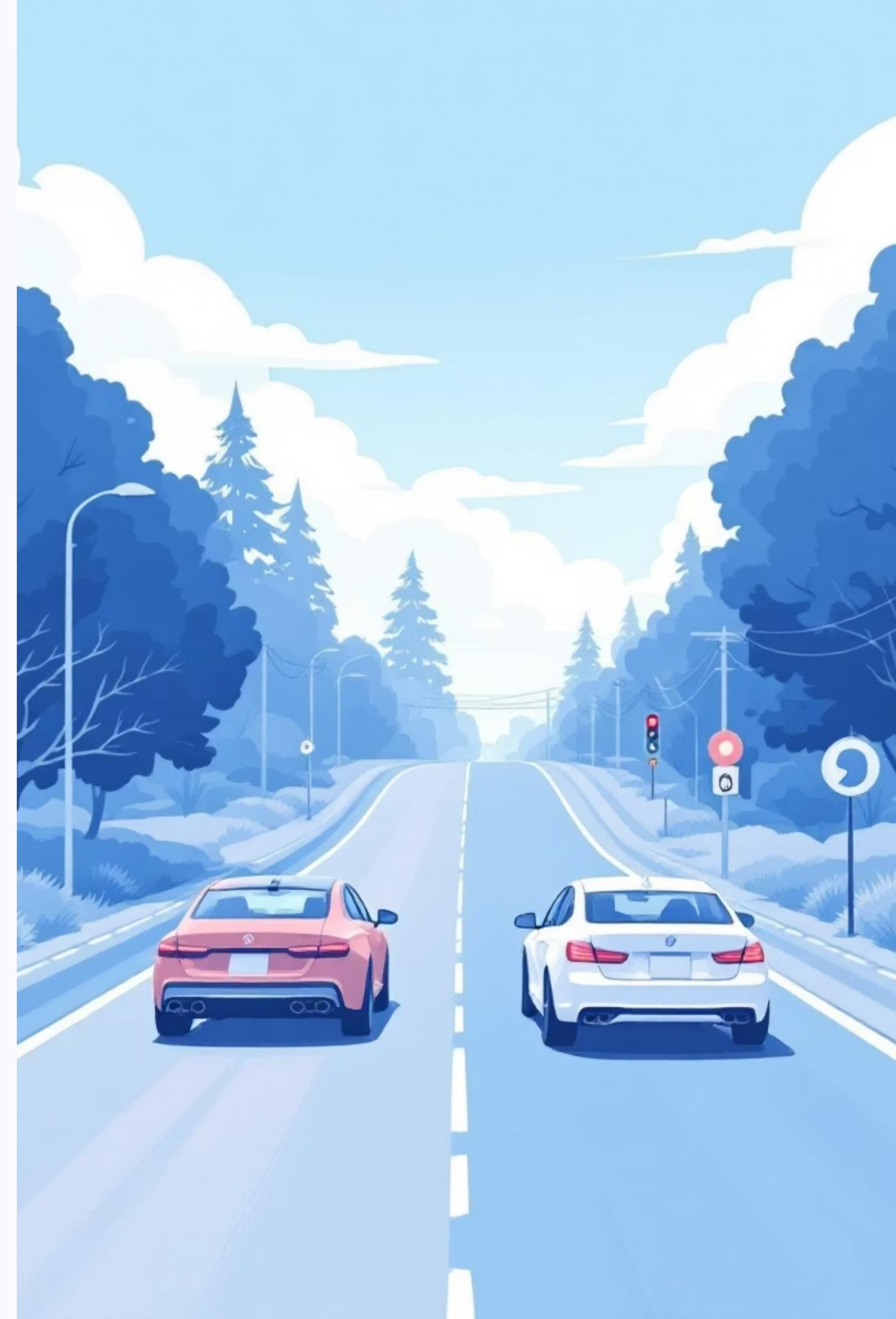
- الأشكال التي تمت اضافتها:

- السيارات: مجموعات من مثلثات تمثل الهيكل، العجلات، والزجاج.
- الطريق: مستطيلات مبنية بمثلثين Two Triangles مع خطوط توقف
- شجرة : مثلثات بسيطة ودوائر عبر Triangle Fan
- إشارة مرورية: دوائر للمصابيح Triangle Fan ومربع للعمود
- استخدام Triangle Fan فعال لرسم دوائر مصابيح الإشارة بدون هندسة مكثفة.

Animation Logic:

المتغيرات الأساسية:

- لتحديد السرعات المتغيرة نستخدم: CarSpeed1- CarSpeed2
- التحقق من الإشارة:
- إذا كانت الحالة أحمر 'R': تتوقف السيارات عند خط التوقف (stopLine)
- إذا كانت الحالة أخضر 'G': تستمر السيارات في الحركة.
- إذا كانت الحالة أصفر 'Y': تتباطئ سرعة السيارات.
- إعادة التدوير: عند خروج السيارة من الشاشة ($car1 > 1.2f$) تعود للظهور من الجهة الأخرى.



التفاعل عبر لوحة المفاتيح



R: إشارة حمراء

تعيين الحالة

`trafficLight =`

`RED`؛ يؤدي إلى توقف

السيارات عند خط التوقف.



G: إشارة خضراء

تعيين `trafficLight`

`GREEN`؛ السماح بعودة

الحركة وفق السرعات

المحددة.



Y: إشارة صفراء

حالة بينية: السيارات تقلل

السرعة تدريجياً قبل قرار

التوقف أو المرور.



H: مصابيح أمامية

للسيارة

1 لتشغيل الأضواء

0 لاطفاء الاضواء

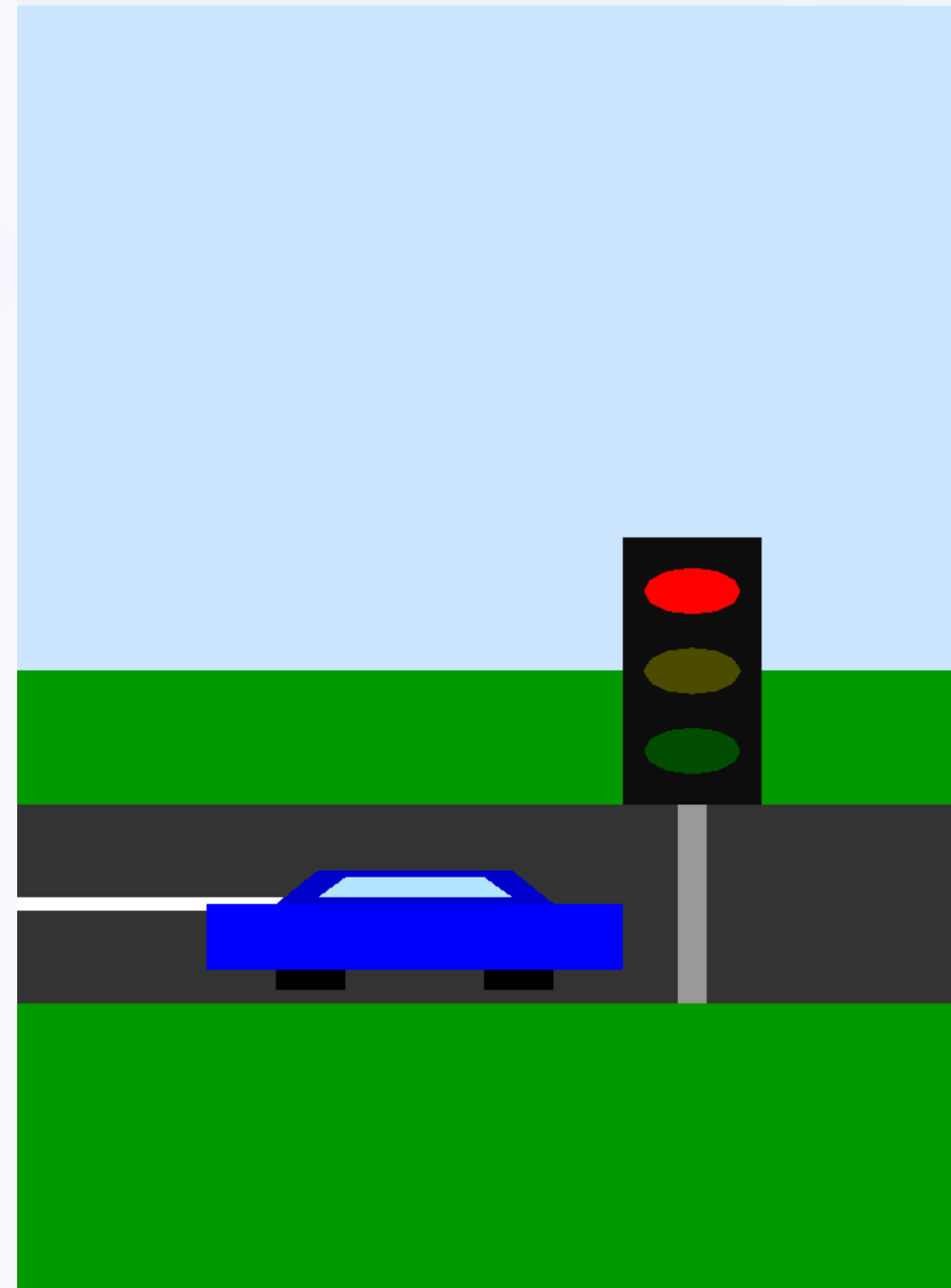
المشاكل والحلول :

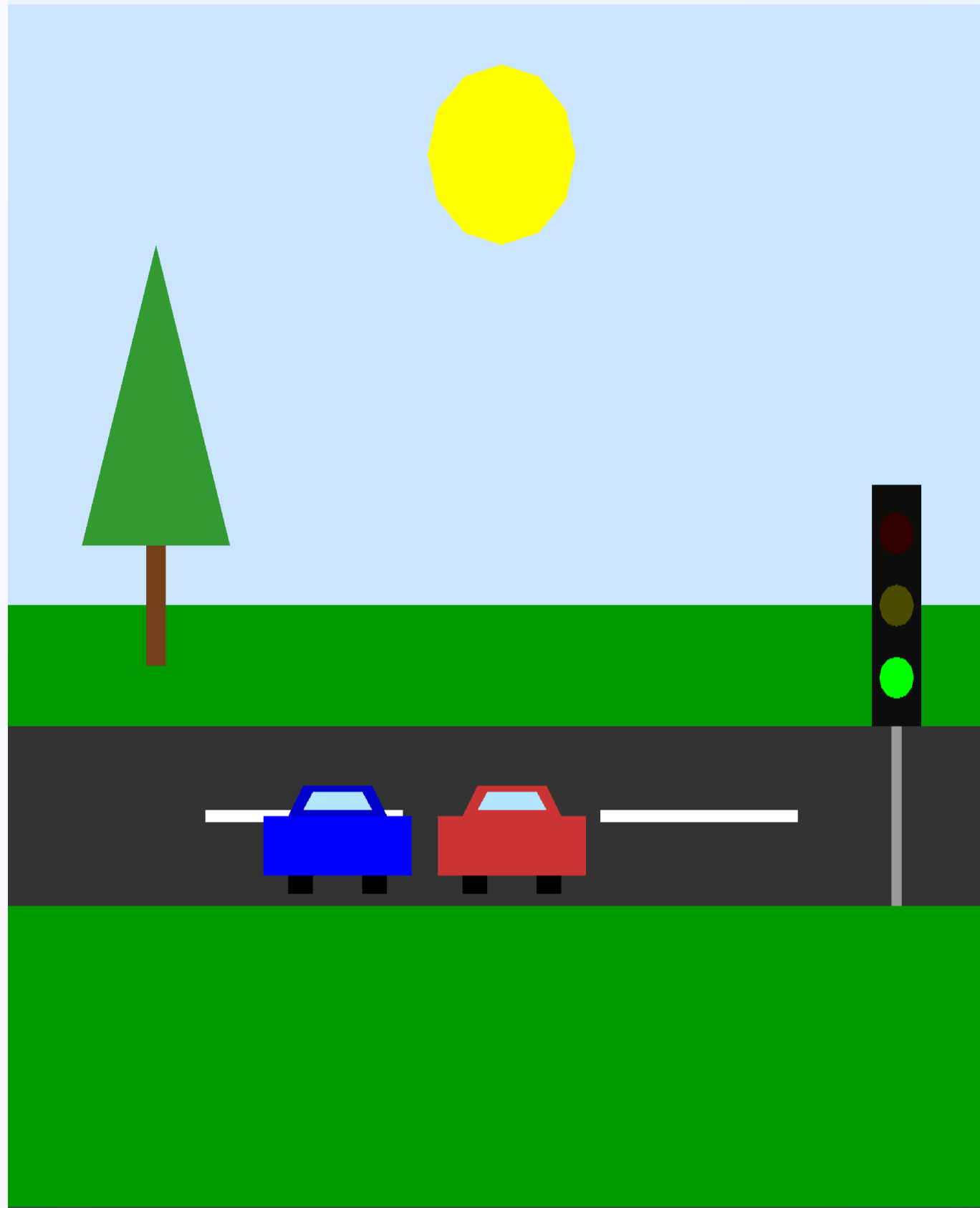
• المشكلة ١ : OpenGL لا يحتوي على دالة جاهزة لرسم الدوائر.

* الحل: بناء دالة رياضية تعتمد على Sin و Cos لتوليد نقاط المحيط يدوياً.

* المشكلة ٢ : مزامنة توقف السيارة مع إشارة المرور بدقة

* الحل: استخدام شروط منطقية تقارن إحداثيات السيارة الحالية بإحداثيات الإشارة وخط التوقف.





في الخاتمة:

- بناء المشهد: تصميم محاكاة مرورية متكاملة باستخدام Modern OpenGL و C++.
- * إدارة الرسومات: نقل البيانات بكفاءة إلى الـ GPU عبر استخدام الـ VBO والـ VAO.
- * التحكم البرمجي: تنفيذ منطق حركة السيارات ومزامنتها مع حالات إشارة المرور (R, G, Y) يدوياً.
- * التفاعلية: ربط مدخلات لوحة المفاتيح لتغيير حالة البرنامج لحظياً (الإشارة والإضاءة).
- * المعالجة المتقدمة: الاعتماد على الـ Shaders والـ Uniforms لضمان أداء عالٍ ومرونة في تلوين الأجسام.
- النتيجة: بيئة مرورية متكاملة تحاكي الواقع وتجمع بين الرسومات والمنطق البرمجي.